

Aegis Care - Comprehensive System Architecture & Technical Documentation

Executive Summary

Aegis Care is an advanced, AI-powered Android healthcare platform designed for continuous elderly fall detection and remote health monitoring. At its core, it seamlessly pairs with an ESP8266-based wearable sensor, leveraging real-time data to provide immediate emergency response. The platform stands out due to its integration with Google's Gemini AI, delivering contextual, conversational health insights, and an immersive AMOLED liquid-glass dynamic user interface.

This document details the full system architecture, hardware-software interaction, and the underlying technology stack used to build Aegis Care.

1. Complete System Architecture

The Aegis Care ecosystem consists of three primary layers:

- 1. Hardware Layer (Wearable Sensor):** Collects physical movement data, detects falls locally, and broadcasts its presence on the network.
- 2. Software Layer (Android App):** Acts as the central hub, continuously monitoring the sensor, processing alerts, displaying an intuitive UI, and managing local data.
- 3. Cloud/AI Layer:** Enhances the app with advanced capabilities, including Gemini AI for health recommendations and Open-Meteo for real-time weather-based advice.

High-Level Architecture Diagram

```
graph TD
    %% Hardware Layer
    subgraph Hardware [Hardware Layer: ESP8266 Wearable]
        MPU6050[MPU6050 Accelerometer/Gyro]
        ESP[ESP8266 Microcontroller]
        MPU6050 -->|I2C Data| ESP
        ESP -->|Detects Fall/Normal| ESP_State[(Local State)]
    end

    %% Software Layer (Android)
    subgraph App [Android Application: Aegis Care]
        Discovery[UDP Discovery Module]
        Monitor[HTTP Polling Engine]
        UI[Dynamic Liquid-Glass UI]
        Theme[Theme Engine]
        Alert[Emergency Alert System]
        Prefs[(SharedPreferences)]

        Discovery -->|Finds IP| Monitor
        Monitor -->|Triggers| Alert
    end
```

```

    Monitor -->|Updates| UI
    UI --> Theme
    Prefs --> UI
end

%% Cloud Layer
subgraph Cloud [Cloud APIs]
    Gemini[Google Gemini AI]
    Weather[Open-Meteo API]
end

%% Interactions
ESP -->|UDP Broadcast :4444| Discovery
Monitor -->|HTTP GET :80| ESP
App <-->|REST/JSON| Cloud

```

2. Hardware Components & Operations

Components Used

- **ESP8266 NodeMCU / Wemos D1 Mini:** A low-cost Wi-Fi microchip with full TCP/IP stack and microcontroller capabilities.
- **MPU6050 Sensor:** Contains a 3-axis accelerometer and a 3-axis gyroscope. It accurately tracks motion, acceleration forces, and orientation.

How the Hardware Interacts with the App

The hardware device acts as a local Wi-Fi server on the same network as the Android device.

Step 1: Auto-Discovery (UDP Broadcast) To prevent the user from manually entering IP addresses, the ESP8266 uses UDP broadcasting. Every 3 seconds, the ESP sends a broadcast packet `ESP_FALL_DETECTOR:192.168.x.x` to port 4444. The Android app listens on this port. Once received, the app extracts the IP address and locks onto the device.

Step 2: Continuous Polling (HTTP) Once discovered, the Android app starts a background thread that sends an HTTP GET request to `http://<esp-ip>/` every 1 second.

- If the MPU6050 detects a sudden spike in acceleration followed by a lack of movement, the ESP sets its state to `FALL`.
- If movement is normal, the ESP state remains `NORMAL`. The Android app reads this response and updates the UI accordingly.

Step 3: State Reset When a fall is detected and the user selects "Mark as Safe" on the app, the app sends an HTTP GET request to `http://<esp-ip>/reset`. This clears the `FALL` state on the microcontroller, resuming standard monitoring.

Hardware-Software Interaction Diagram

```

sequenceDiagram
    participant App as Aegis Care App
    participant ESP as ESP8266 Sensor
    participant MPU as MPU6050

```

```
Note over App,ESP: 1. Auto-Discovery Phase
loop Every 3 seconds
    ESP->>App: UDP Broadcast (Port 4444): "ESP_FALL_DETECTOR:192.16
end
App-->>App: Locks ESP IP Address

Note over App,ESP: 2. Monitoring Phase
loop Every 1 second
    App->>ESP: HTTP GET /
    ESP->>App: 200 OK: "NORMAL"
end

Note over ESP,MPU: 3. Fall Event
MPU->>ESP: High Acceleration + Impact detected
ESP-->>ESP: State = FALL

App->>ESP: HTTP GET /
ESP->>App: 200 OK: "FALL"

Note over App: App triggers Alarm, Vibration, UI Alert

Note over App,ESP: 4. User Resolution
App->>ESP: HTTP GET /reset
ESP-->>ESP: State = NORMAL
ESP->>App: 200 OK
```

3. Technology Stack

Android Application

- **Language:** Java
- **UI Framework:** Standard Android XML combined with advanced Custom Canvas Views (for particle systems, animated graphs, and risk meters) and GradientDrawable for the Liquid-Glass aesthetic.
- **Persistence:** SharedPreferences for storing user profiles, custom alarm URIs, and theme preferences.
- **Networking:** HttpURLConnection for REST APIs and polling, DatagramSocket for UDP discovery.

External APIs

- **Google Gemini API:** Provides the conversational AI persona ("Aegis AI"). It generates dynamic health reports and interactive chat responses based on the user's fall history and medical condition.
 - **Open-Meteo API:** A free weather API providing real-time conditions used to generate physics-based particle weather animations on the app dashboard.
-

4. Key Software Features & Engineering

4.1 The Dynamic Theme Engine

Aegis Care implements a system-wide theme engine allowing users to select from 10 distinct "Neon" themes (e.g., Cyan, Crimson, Midnight Gold). When a theme is selected, the app programmatically repaints all `GradientDrawable` backgrounds, toolbar strokes, icon tints, graph accents, and glowing shadows. The transition is managed by a 400ms alpha fade animation, ensuring a smooth visual shift.

4.2 Emergency Alert UX

When `FALL` is polled from the ESP, the app enacts an aggressive yet visually premium response:

1. Loops a high-volume alarm (either default or user-selected).
2. Vibrates the device using the hardware `Vibrator` service.
3. Slides up a custom liquid-glass Bottom Sheet presenting three options: **Call Emergency**, **View Member Profile**, or **Mark as Safe**.
4. Initiates a 4-second programmatic cooldown (`safeCooldownUntil`) after "Mark as Safe" to prevent immediate re-triggering of the alarm due to network lag.

4.3 AI Health Report & Typewriter Engine

The AI Health report utilizes a custom-built handler to perform a typewriter animation for AI text. Instead of blocking the UI thread with `Thread.sleep()`, it uses `Handler.postDelayed(14ms)` to append characters one by one. The parent layout uses `WRAP_CONTENT`, meaning the card organically expands downwards as the AI "types" out the patient's stability summary.

5. Security & Fallbacks

- **Thread Safety:** The network polling, UDP listening, and HTTP reset calls (`sendReset()`) are strictly segregated from the Main UI thread to prevent `NetworkOnMainThreadException` and ensure smooth 60fps animations.
- **Network Resilience:** If the ESP8266 changes IP (DHCP lease renews) or disconnects, the HTTP poll times out after 2 seconds, returns an "ERROR" state, and immediately spins up the UDP Discovery thread to find the sensor's new IP.

Generated for the Aegis Care Development Team.